Pairwise Siamese Transformer 모델 매뉴얼

본 매뉴얼은 금융 전략 비교를 위한 Pairwise Siamese Transformer 모델의 전반적인 아키텍처, 데이터 파이프라인(독립 및 종속 변수), 하이퍼파라미터 최적화 원리 및 모델의 의의를 상세히 설명합니다.

1. 전반적인 프로젝트 설명

이 프로젝트는 전통적인 시계열 가격 예측(Price Prediction)에서 벗어나, **동일한 시장 환경 하에서 두 가지 다른 금융 투자 전략의 행동(Behaviour) 차이를 모델링**하는 것을 목표로 합니다. 주어진 상황에서 각 전략이 어떻게 반응하고 포지셔닝하는지 행위적 특징(Feature)을 추출하고, 이를 기반으로 어떤 전략이 더 우수한 성과를 낼지 분류하는 딥러닝 아키텍처입니다.

---

2. 변수 정의 및 모델 아키텍처 (데이터 흐름)

2.1. 독립 변수 (입력값, Independent Variables)

모델에 입력되는 독립 변수는 특정 시장 상황 속에서 두 전략이 취한 다차원 시계열 데이터 시퀀스입니다. 핵심은 시장 데이터뿐만 아니라 **각 전략이 당시에 취했던 포지셔닝 상태가 독립 변수에 포함**된다는 점입니다.

- **기초 시장 환경 데이터:** Open, High, Low, Close (고가, 저가, 시가, 종가), Volume (거래량) - (두 전략에게 공통으로 주어지는 환경)
- **전략의 고유 행동/상태 데이터: Position (해당 시점의 포지션 상태)**, Log Return (로그 수익률: 스케일 독립적인 수익 지표), Proxy Return (마이크로 모멘텀 및 방향성을 나타내는 파생 수익 지표) - (두 전략이 서로 다르게 띠는 고유 상태)

*참고: 입력 전 모든 수치형 독립 변수는 시퀀스 단위로 **Z-score 정규화(Normalization)**를 거쳐 절대적 크기보다는 상관관계 및 행동 패턴에 모델의 어텐션(Attention)이 집중되도록 합니다.*

2.2. 종속 변수 (출력값, Dependent Variables)

모델의 최종 출력값은 **이진 분류(Binary Classification) 값 (0 또는 1)**입니다. 두 개의 전략(Strategy A, Strategy B) 중 어느 전략이 주어진 기간 내에 더 좋은 성과(우위)를 거두었는지를 나타내는 라벨링 결과(예: A가 우위면 0, B가 우위면 1 등)입니다. 성능 비교의 정답지 역할을 합니다.

2.3. 독립 변수와 종속 변수의 처리 및 흐름 구조

데이터 사이언스 및 딥러닝 모델링 측면에서, 독립 변수와 종속 변수가 어느 단계에서 어떻게 분리되어 모델에 적용되는지 명확히 이해하는 것이 중요합니다.

1. **독립 변수 (Features)의 모델 투입과 차원(Dimension) 변환:** 모델에 들어가는 원시 데이터는 시간의 흐름을 담은 2차원 행렬(Matrix) 형태입니다. 예를 들어, 과거 100일간의 데이터를 보고 5개의 변수(시가, 종가, 거래량, 포지션, 수익률 등)를 사용한다면 입력 데이터의 형태는 `[Sequence Length(100), Feature Dimension(5)]` 의 크기를 갖는 2D 텐서가 됩니다.

   이 데이터(`seq_A` , `seq_B`)는 시퀀스 단위로 Z-score 정규화를 거친 후 **오로지 이 형태 그대로 Transformer Encoder의 유일한 입력(Input) 값으로 투입**됩니다. 인코더 내부를 거치며 시간축(100일)에 존재하는 정보들이 서로 어텐션(Attention) 교환을 수행하고, 마지막 평균 풀링(Mean Pooling) 단계를 지나면 최종적으로 이 2D 텐서 행렬은 각 전략의 전반적인 행동 특성만을 압축하여 담아낸 `[256]` 크기의 1차원 단일 벡터(1D Vector)로 차원이 압축 및 변환됩니다.

2. **종속 변수 (Labels)의 분리 위치 (훈련 루프의 목적 함수):** 어느 전략이 우세한지에 대한 정답 라벨인 종속 변수(0 또는 1)는 데이터 유출(Data leakage)을 막기 위해 **모델(인코더) 내부에 절대 입력 변수로 들어가지 않습니다.** 종속 변수는 훈련(Training) 과정의 최후단에서만 등장합니다. 아키텍처가 독립 변수만을 가지고

도출해낸 예측 확률값 출력(Output)과 실제 정답인 종속 변수를 비교하여 **Binary Cross-Entropy (이진 교차 엔트로피) 손실 함수를 계산**하는 데 사용되며, 오차를 줄이기 위한 역전파(Backpropagation) 과정에서 모델의 가중치를 업데이트하는 데 쓰입니다.

3. **추론 및 반환(Inference & Return) 단계:** 결과적으로 학습이 완료된 후 실전(Test) 단계에서는 종속 변수의 존재 없이, 인코더가 오로지 독립 변수만 입력받고 내부 가중치 연산을 거쳐 "두 전략 중 어떤 전략이 더 적합한가"에 대한 확률값(0~1)을 최종적으로 변환 및 반환합니다.

2.4. 변환 및 출력의 세부 임베딩 아키텍처 (Architecture)

모델은 종속 변수를 완벽히 배제한 채 입력 데이터 시퀀스(독립 변수)를 받아 다음과 같은 아키텍처 설계 과정을 거쳐 최종 결과를 도출합니다.

1. **상태 임베딩 추출 (Sequence Encoder)**
   - **과정:** 두 전략의 데이터 시퀀스(`seq_A` , `seq_B`)는 각각 **8-Layer Transformer Encoder** 구조를 통과합니다. 구체적으로는 각 시점의 피쳐가 **256차원의 벡터 공간(Model Dimension = 256)** 으로 투영된 후, **4개의 멀티 어텐션 헤드(N_Heads = 4)** 를 갖춘 8개의 레이어를 거치게 됩니다. 이 과정에서 Dropout(0.1), Layer Normalization, Residual Connection이 차례로 적용되어 과적합을 방지하고 안정적인 학습을 보장합니다.
   - **수행 내용:** 트랜스포머의 어텐션 메커니즘이 시점 간의 상관관계와 비지역적(Non-local) 행동 특징을 추출합니다. 이후 추출된 전체 시퀀스 토큰(각 시점의 특징 벡터들)을 시간 길이에 대해 모두 더해 평균내는 **평균 풀링(Mean Pooling)** 과정으로 이어집니다. 결과적으로 복잡한 다차원(2D 텐서) 시계열 데이터가 각 전략의 전체적인 속성을 요약하는 **256차원 크기의 1D 단일 행동 임베딩 벡터(`z_A` , `z_B`)** 로 가공 및 압축되어 추출됩니다.
2. **샴 네트워크 구조 (Siamese Design)의 필요성**
   - **과정:** 전략 A와 전략 B를 처리하는 각각의 Transformer 인코더는 **가중치를 완벽하게 공유(Weights Sharing)**하는 샴 네트워크(Siamese Network) 구조를 가집니다.
   - **왜 단일 인코더(Single Encoder)를 쓰지 않는가?:** 시장 환경(OHLCV) 변수는 동일하지만, 각 전략이 그 시점에 취한 **포지션(Position)과 파생 수익률(Log/Proxy Return)이라는 상태 변수는 전혀 다릅니다.** 따라서 `seq_A` 와 `seq_B` 는 엄연히 서로 다른 데이터 시퀀스입니다. 만약 이 두 시퀀스를 단순히 이어 붙여(Concatenation) 하나의 거대한 트랜스포머에 통과시킨다면, 어텐션 모델은 A전략의 특징과 B전략의 특징이 서로 섞이는 것을 방지하기 위해 불필요하게 복잡성을 키우게 되며, 파라미터 낭비 및 심각한 과적합(Overfitting)을 초래합니다.
   - **의미:** 독립적인 두 개의 입력부를 가지되 가중치를 공유하는 샴 네트워크를 사용하면, 모델은 전략 A와 전략 B를 '동일한 잣대(동일한 행동 언어 공간)'로 평가하도록 강제됩니다. 이는 파라미터를 절반으로 줄이면서도 두 전략을 일관성 있게 비교(Apples-to-apples comparison)할 수 있도록 만드는 핵심 기법입니다.
3. **비교 헤드 (Comparison Head) 및 중간 병합**
   - **과정:** 두 개의 추출된 임베딩 벡터는 단순 연결을 넘어 다음과 같은 형태로 구성(Concatenation)됩니다. `[z_A, z_B, |z_A - z_B|]`
   - **의미:** 이 결합 방식은 각 전략의 절대적 행동(`z_A` , `z_B`)뿐만 아니라 두 전략 간의 상대적인 차이/불일치성(`|z_A - z_B|`)을 모델에 명시적으로 제공하여 비교 능력을 극대화합니다.
4. **최종 출력 (MLP & Sigmoid)**
   - 위 결합 벡터(길이 768)는 3개의 선형 층으로 구성된 MLP 알고리즘 (768 → 256 → 128 → 1)을 통과합니다.
   - 마지막으로 Sigmoid 활성화 함수를 지나 0과 1 사이의 확률값으로 변환되며, 둘 중 어느 전략이 우세한지에 대한 최종 결정을 내립니다.

---

3. 모델의 우수성 및 하이퍼파라미터 선택의 최적 결과 이유

3.1. 이 모델이 기존 모델 대비 좋은 점

- **전통적인 RNN/LSTM의 한계 극복:** 순환신경망은 긴 시퀀스를 지날 때 노이즈가 누적되거나 과거 기억이 희석되지만, Transformer 기반 본 모델은 **어텐션(Attention) 메커니즘**을 통해 과거의 중요했던 시장 국면과 현재 전략의 반응을 다이렉트로 매핑하여 다중 스케일적인 특성을 잡아냅니다.
- **Siamese Network 비교의 정밀성:** 단순히 특징을 추출해 합치는 것이 아니라 가중치를 공유하는 인코더가 작동하므로, 두 금융 전략이 전혀 다른 잣대로 평가받는 현상(Overfitting)을 원천적으로 막고, 추론 성능의 신뢰성을 극대화했습니다.

3.2. 하이퍼파라미터 제어 및 피쳐 엔지니어링 최적화

이 모델이 달성한 81%라는 높은 Test Accuracy는 절제된 피쳐 설정과 안정성을 중시한 학습 파라미터 조합의 결과물입니다.

- **최소주의 피쳐 엔지니어링(Lean Features):**
  - MACD, RSI, ATR과 같은 복잡한 보조지표들을 실험했으나 **모두 제외**했습니다.
  - *이유:* 어텐션 기반 트랜스포머는 그 자체로 강력한 패턴 인식력을 가집니다. 이미 가공된 보조지표를 추가했더니 모델이 단기 변동(Short-term swings)에 너무 과민하게 반응하여 오히려 노이즈가 증폭되었습니다. 대신 기초 변수와 'Log Return(로그 수익률)' 등 최소한의 변수만 투입하여 모델이 자체적으로 본질적 행동에 집중하도록 설계하여 최상의 임무 수행력을 끌어냈습니다.
- **안정적인 학습률 (Learning Rate = 1e-4):**
  - 노이즈가 강한 금융 데이터와 파라미터가 많은 트랜스포머의 결합은 종종 학습 불안정으로 이어집니다. 다소 보수적인 학습률(`1e-4`)과 Adam 최적화의 결합은 어텐션 값이 튀는 것을 막고 손실 함수(Loss)가 부드럽게 수렴하도록 만들었습니다.
- **모델 복잡도의 스위트스팟 (8 Layers / 4 Heads / 256 Dim):**
  - 은닉층 차원과 레이어 수를 과도하게 늘리지 않았습니다. 금융 시계열 전략의 비교에 있어, 너무 깊은 신경망은 훈련 데이터의 사소한 마이크로스트럭처 구조까지 외워버립니다. 현재의 설정은 행위 군집화(Behavioural clustering)를 수행하는 데 있어 과적합을 피하는 가장 최적의 비율로 입증되었습니다.

---

4. 이 모델이 가지는 의의

1. **투자 접근법의 패러다임 전환:**
   - "미래 가격이 얼마인가?"라는 맞추기 힘든 정답지측 예측(Forecasting) 프레임에서 벗어나, "이 전략이 과거 비슷한 국면에서 어떻게 행동했는가?"라는 **행동 패턴 중심 의사결정 모델(Representation Learning)**로 전환했다는 데 큰 의의가 있습니다.
2. **행동 지문(Behavioural Fingerprints)의 정량화:**
   - 단지 과거 수익률(리턴)이라는 단편적인 결과물만 보는 백테스트의 한계를 넘습니다. 변동성 국면이나 특정 시장 마찰 시기에 전략들이 어떻게 방어하고 공격했는지, 그 '과정적 특이성(행동 지문)'을 모델이 스스로 인식하고 정량화해냅니다.
3. **확장 가능성 기반 마련:**
   - 학습된 인코더 모델은 향후 수많은 전략들을 다차원 공간에 군집화(Clustering)하거나, 대안적 포트폴리오를 탐색할 때 강력한 사전 훈련(Pre-trained) 기반으로 활용될 잠재력을 가져 의미가 큽니다.